New Horizons
Centro de Capacitación

Práctica: Endpoint de Login, Password Hashing y Protección con [Authorize]

Paso a paso — Configurar AddJwtBearer en Program.cs, controlador AuthController y prueba en Swagger

Curso	Desarrollo Full Stack con C# y Angular
Sesión	Sesión 13 — Seguridad y Autenticación Backend con JWT en ASP.NET Core
Docente	Edwin Barrientos
Duración estimada	75–90 minutos
Nivel	Intermedio (aplicación práctica)
Entorno	.NET 8 SDK, Visual Studio 2022 / VS Code, Angular 18 Standalone, Bootstrap 5

1. Objetivos de la Práctica

- Implementar el algoritmo de Password Hashing criptográfico con PBKDF2.
- Crear AuthController con el endpoint público POST /api/auth/login.
- Registrar los esquemas de autenticación y autorización en el pipeline de .NET 8.
- Blindar los controladores de Solicitudes con el atributo [Authorize].
- Configurar Swagger para permitir autenticación Bearer y probar peticiones protegidas.

2. Desarrollo Paso a Paso

Paso 1 — Configurar el Middleware de Autenticación en Program.cs

Añadimos la configuración de JwtBearer al contenedor de inyección de dependencias:

src/SolicitudesApp.Api/Program.cs

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidAudience = builder.Configuration["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!))
        };
    });

app.UseAuthentication();
app.UseAuthorization();
```

Paso 2 — Crear el AuthController

Exponemos el endpoint de inicio de sesión validando credenciales y retornando el token JWT:

src/SolicitudesApp.Api/Controllers/AuthController.cs

```
[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly IAuthService _authService;
    private readonly IUsuarioRepository _usuarios;

    public AuthController(IAuthService authService, IUsuarioRepository usuarios)
    {
        _authService = authService;
        _usuarios = usuarios;
    }

    [HttpPost("login")]
    public async Task<ActionResult<AuthResponseDto>> Login([FromBody] LoginRequestDto dto)
    {
        var usuario = await _usuarios.ObtenerPorEmailAsync(dto.Email);
        if (usuario == null || !_authService.VerificarPassword(dto.Password, usuario.PasswordHash, usuario.PasswordSalt))
            return Unauthorized(new { mensaje = "Credenciales de acceso incorrectas." });

        var token = _authService.GenerarToken(usuario);
        return Ok(new AuthResponseDto(usuario.Id, usuario.NombreCompleto, usuario.Email, usuario.Rol.Nombre, token,
DateTime.UtcNow.AddHours(8)));
    }
}
```

Paso 3 — Proteger SolicitudesController con [Authorize]

Añadimos [Authorize] al controlador para restringir el acceso a usuarios con token válido:

src/SolicitudesApp.Api/Controllers/SolicitudesController.cs

```
[Authorize]
[ApiController]
[Route("api/[controller]")]
public class SolicitudesController : ControllerBase
{
    // Todos los endpoints de consulta y creación requieren token Bearer
}
```

> **Validación de la Práctica:** Asegúrese de compilar y ejecutar las pruebas para validar que los endpoints y vistas operen conforme a los estándares corporativos solicitados.
